aero_auto 1.0 User Manual

Dark edition - run the GUI, schedule campaigns, read the reports.

The practical guide: what you need, how to install it, how to run every tool from the GUI or the command line, and what lands in the report folder. Engineers looking for the "why" behind the code should open the technical guide.

Contents

1 Overview

2 Requirements and dependencies

3 Install

4 Run

5 The GUI, tab by tab (run_ui.sh / aero_ui.exe)

6 File locations at a glance

7 Environment overrides

8 What a campaign writes

9 The report folder

10 Regulatory reference

11 Packaging and platform notes

12 Changelog highlights

1 Overview

Module	Purpose
aero_auto.py	core engine - rules, geometry, JavaFoil, planner
aero_targets.py	CL*A / CD*A target report
aero_report.py	ranked tables, workbook, graphs, --resim
flowfield.py	schematic potential-flow model for field images
aero_ui.py	the GUI shell
render_field.py	headless field PNG renderer
launcher.py	one entry point for every tool

2 Requirements and dependencies

Python packages (requirements.txt): numpy, pandas, matplotlib, openpyxl, Pillow (icon build); pyinstaller is only needed for standalone builds.

System tool	Needed for	Notes
Python 3.9+ with Tkinter	everything; the GUI uses Tk	python.org installer bundles Tk on Windows
Java 8+ (java)	simulation	javafoil.jar required in JAVA_DIR
faketime	Linux only	skipped automatically on Windows (2.3 of the guide)
javafoil.jar + mhclasses.jar	simulation	~/ MH-AeroTools / JavaFoil / - verified present
airfoil .dat library	geometry and polars	override with AERO_AIRFOILS

3 Install

```
python3 -m pip install -r requirements.txt

# optional standalone (no Python on the target machine):
python3 create_standalone.sh     # linux/macos -> dist/aero_ui/
create_standalone.bat            # windows     -> dist\aero_ui\aero_ui.exe
```

4 Run

GUI

```
./run_ui.sh            # linux/macos
run_ui.bat             # windows
# or
python3 launcher.py ui
```

Headless tools

```
python3 launcher.py campaign --role fw --flow ground --height 300 --hours 0.5
python3 launcher.py report   --results ~/aero_campaign/results.csv --top 3
python3 launcher.py report   --results .../ground/h300pct/results.csv --resim
python3 launcher.py targets  --results ~/aero_campaign/results.csv --role fw
python3 launcher.py field    --main s1223 --flap e423,30,-6,100,5 --out f.png
                              --ground-height 300
```

5 The GUI, tab by tab (run_ui.sh / aero_ui.exe)

Five tabs wrap the engine: Configure, Airfoils, Campaigns, Results and Graphs. Launch the GUI with **run_ui.sh** on Linux, **run_ui.bat** on Windows, or simply double-click **aero_ui.exe** in the standalone distribution (dist/aero_ui/). The same engine runs behind the window, so a config you have just sanity-checked can be simulated or campaigned without ever leaving the app. Every JavaFoil operation runs in a worker thread - the window never freezes while a polar is computed.

5.1 Configure tab

Control	What it does
Main element	the airfoil that defines the wing; the combo lists every profile found in your .dat folder
Role: front / rear / custom	sets the span used by CL*A and the force model (fw 1400 / rw 950 / custom 1000 mm) and the FSUK/FSG rule set that is checked
Main chord (mm)	physical chord in millimetres; enables the all-mm geometry columns and the force conversion
Speed (m/s)	freestream speed for q = 0.5 * rho * V^2 in the force model
Reynolds number	run Reynolds number sent to JavaFoil (default 570 000)
Alpha start / end / step (deg)	polar sweep angles; the default -20..+5 step 1 covers the stall search
Flap rows F1..F3	per slot: airfoil combo + S% scale of main chord, A deg mount angle, X% position, Y% position; tick the checkbutton to enable the row
enforce FSUK/FSG size limits	master switch for the regulatory checks (stack height/length, ground clearance)
Ground height (% chord)	wing height above the ground for ground-effect runs; leave empty or "none" for unbounded flow
Sanity check geometry	instant PASS/FAIL against the size limits - prints the exact mm value vs limit and the per-pair gap/overlap in %, then redraws the preview (green PASS / red FAIL, ground line drawn when a ground height is set)
Simulate this wing now (JavaFoil)	runs the real JavaFoil analysis in a worker thread, appends the row to <OUT_DIR>/results.csv and logs Run_ID, Status, Cl@0, Cd@0, max L/D, stall angle and Downforce_N
Simulate in Terminal	launches the simulation inside an external terminal (konsole, alacritty, gnome-terminal, xterm) for real-time stdout tracking
Batch Simulate	sweeps multiple wing configurations or angle cases sequentially in worker threads
Extract JavaFoil Graphs (Native)	runs JavaFoil headlessly and extracts all 6 native plots (drag polar, lift curve, moment curve, surface velocity/Cp distribution, skin friction, and boundary layer thickness) straight to the Graphs tab
status log + geometry preview	right-hand pane: live log of every action and an embedded interactive matplotlib drawing of the element stack scaled to mm with ground line indicator

5.2 Airfoils tab

Control	What it does
store location label	shows the internal airfoil store folder <APP_DATA_DIR>/airfoils (default ~/.aero_auto/airfoils), auto-created on first use
Import .dat...	copies any airfoil .dat file into the internal store under the name you type; the profile is then available in every combo - you no longer have to remember which folder holds a profile
Remove selected	deletes the selected stored profile from the internal store (only affects the store, never your original .dat files)
Open store folder	opens the store folder in the file manager when you want to inspect or manually drop files in
Refresh	re-scans the store (and AIRFOIL_DIR) in case files changed on disk
store list	shows every profile currently inside the internal store with its full path; select a row to remove it

5.3 Campaigns tab

Control	What it does
Hours	campaign time budget for the planner (can be small, 0.1 works)
Workers	number of parallel JavaFoil worker directories
Pool	comma-separated airfoil names restricting exploration to profiles you actually have .dat files for
Start campaign	runs the 3-phase planner; progress streams into the log; rows are written to <OUT_DIR>/results.csv
live stats label	during a campaign shows the elapsed time, how many new simulations have been finished and the current runs/second - you can see the campaign progressing without counting rows
Write report	builds the full report folder (Best/, graphs/) for <OUT_DIR>/results.csv without leaving the app

5.4 Results tab & CSV Dataset Navigator

The Results tab is a universal dataset browser and configuration inspector that navigates across all simulation and optimization files (results.csv, iteration_sheet.csv, top_results.csv, ground-effect sweeps, and custom CSVs).

Component	Capabilities and Workflow
CSV Dataset Navigator	Left sidebar recursively discovering all CSV files across ~/aero_campaign, ground/ folders, the workspace root, backups, and custom user folders. Displays classified tree with category badges, row counts and file sizes. Includes live search filter, 1-click Quick Switch combo, 'Browse Any CSV...' file picker, and 'Open Ext ■' spreadsheet launcher.
Universal Dynamic Table	Automatic schema detection for arbitrary CSVs (iteration sheets, optimization logs) without hardcoded limits. Prioritizes primary metrics (Run_ID, Downforce, Drag, Max_LD, Cl_a0, Cd_a0), features numeric-aware click-to-sort headers (▲/▼), real-time global multi-column search, Status filter (Valid/Failed/custom), Tier filter (1/2/3), alternating row striping, status color badges, and 'Export Filtered CSV...'.
Dynamic KPI Banner	Real-time summary strip calculating active/filtered statistics: Rows count (e.g. 1,420 / 23,272), Max Downforce (supporting both N and CL units), Max L/D efficiency ratio, Best Score, and Valid % legality compliance.
Run Inspector	Dual-view inspection: formatted Aerodynamic Telemetry Card (Run ID, Status, Elements, Polars, Forces, and mm geometry) plus a comprehensive Property Grid listing every key-value attribute of the selected run.
Action Workflows	'Load to Configure': pushes selected profile and flap coordinates (scale, angle, X, Y) to Configure tab and triggers instant sanity check and preview redraw. 'Send to Optimizer': passes selected configuration as baseline for automated sweeps. 'Graphs for selected run': renders custom polar, geometry, and flowfield PNGs, plus headlessly extracts all 6 authentic JavaFoil native graphs, and opens Graphs tab.
Raw CSV Data Editor	Dedicated sub-tab displaying the raw text representation of the active CSV file, with line and character counters and in-place 'Save CSV Changes' with automatic cache reloading.

5.5 Graphs tab

The Graphs tab is the image side of the app. It discovers every results file on disk - the main one plus each ground-effect folder - and any generated graph images, previewing them right inside the window.

Control	What it does
results file combo	lists <OUT_DIR>/results.csv, each ground folder's results.csv and the file you last loaded; pick one to inspect that campaign's data
Write report	builds the full report folder for the selected results file (also used to update the images after a new campaign)
Refresh	re-scans the selected results folder for polar, geometry, field, and native javafoil_* PNGs
Open in viewer	opens the current image in your system image viewer

Control	What it does
image list	every <results-dir>/graphs/*.png found for the chosen results file (including custom and native JavaFoil plots)
preview	click an image name to see it scaled inside the app (needs Pillow; without it the list still works, just no inline preview)

6 File locations at a glance

Where airfoils are read from and every kind of output lands. All paths can be moved with the environment overrides in chapter 7.

Artifact	Location
airfoil .dat files	default AIRFOIL_DIR "/mnt/data/FSEA/AIRFOILS JAVAFOIL/AIRFOILS" - put one .dat per profile in this folder (top-level files only, no sub-folders). The GUI combos are built from these filenames. Set AERO_AIRFOILS to point elsewhere
single simulations / GUI + CLI campaigns	<OUT_DIR>/results.csv (default ~/aero_campaign/results.csv), appended after every run
ground-effect campaigns (CLI)	<OUT_DIR>/ground/<height>pct/results.csv - one file per ride height
ranked tables	<results-dir>/Best/configs.csv, Best/overview.txt, Best/overview.xlsx (next to whichever results.csv you reported)
readable CSV + legend	<results-dir>/results_human.csv and <results-dir>/columns_legend.csv
polar and geometry graphs	<results-dir>/graphs/polar_<Run>.png and geometry_<Run>.png
native JavaFoil graphs	<results-dir>/graphs/javafoil_polar_cl_cd_<Run>.png, javafoil_polar_cl_alfa_<Run>.png, javafoil_polar_cm_alfa_<Run>.png, javafoil_velocity_cp_<Run>.png, javafoil_bl_cf_<Run>.png, javafoil_bl_delta_<Run>.png - native headless captures from JavaFoil's internal cards
velocity maps (field sims)	<results-dir>/graphs/field_<Run>.png - schematic potential-flow velocity field for that config; the render_field.py CLI produces a single picture from any configuration
target report	targets_report.csv next to the results file you pointed the tool at

Note. The polar graphs are the authoritative data. field_*.png images are a schematic potential-flow visualisation whose circulation comes from each config's own JavaFoil Cl - not a second force solver.

7 Environment overrides

Variable	Effect
AERO_JAVA_DIR	folder that contains javafoil.jar
AERO_AIRFOILS	folder with the .dat profiles
AERO_OUT	campaign output folder (default ~/aero_campaign)
AERO_ICON	explicit GUI icon path

8 What a campaign writes

~/aero_campaign/results.csv holds unbounded runs; ground-effect runs land in ~/aero_campaign/ground//results.csv. Rows carry the configuration, the polar metrics, per-pair geometry, forces and quality:

```
Run_ID Phase Main F1_Airfoil F1_Scale F1_Angle F1_X F1_Y F2_* F3_* \
Cl_peak Cd_peak Stall_Angle Max_LD Cl_a0 Cd_a0 Cm_a0 Cl_am2 Cl_ap2 \
Downforce_N Drag_N Tier Score Status  + per-pair gap/overlap/step + mm geometry
```

9 The report folder

File	Content
Best/configs.csv	the top-N, round-robin across stack sizes
Best/overview.txt	human summary incl. spans, q and force model
Best/overview.xlsx	Top configs / All rows / Legend sheets
results_human.csv	every valid row, readable column names
columns_legend.csv	column - unit - description for everything
graphs/*.png	polar, geometry, field, and native JavaFoil engine graphs (polars, Cp, BL) per config

Note. Polar graphs are the authoritative data. field_*.png images are a schematic potential-flow visualisation, not a second force solver.

10 Regulatory reference (FSUK 2026 / FSG v1.1)

Rule	Limit
Min ground clearance	35 mm
Front wing max stack height	250 mm
Front wing max stack length	600 mm
Rear wing max total height	1100 mm
Rear wing max stack length	250 mm
Front wing span	1400 mm
Rear wing span	950 mm
Targets CL*A / CD*A roof	fw 1.75 / 0.45 - rw/beam 1.85 / 0.95 (tol ±0.4)

11 Packaging and platform notes

Four distributions come from the same modules: the Linux tar.gz, the Windows zip, the installed desktop app (~/Applications/aero_auto, registered via the .desktop entry), and the standalone aero_ui.exe built with PyInstaller under wine. The exe bundles assets and docs with --add-data. Rebuild with create_standalone.bat on Windows or the wine command in the technical guide's packaging chapter.

12 Changelog highlights

- regulatory engine + GUI verdict, geometry_ok gate before JavaFoil
- ground-effect campaigns and per-height result files
- force model (Downforce_N / Drag_N) and all-mm geometry columns
- target report against CL*A / CD*A roofs
- calibrated potential-flow model (replaces the abandoned linear solver)
- full report pipeline: workbook, legend, duplicate-safe headers, --resim
- threaded Tk GUI with a queue-based thread bridge and icon support
- cross-platform packaging: launcher, field CLI, desktop app, wine-built exe
- Results tab CSV Dataset Navigator with recursive multi-folder discovery and live filter
- universal dynamic table supporting arbitrary CSV schemas with numeric sorting (▲/▼)
- real-time aerodynamic KPI banner (Rows, Max Downforce, Max L/D, Best Score, Valid %)

dual-view run inspector (aero telemetry card + property grid)

1-click engineering handoff: 'Load to Configure' and 'Send to Optimizer'

raw CSV data viewer and editor with live line/char stats and in-place save

aeronautical dark UI theme polish and multi-terminal simulation launcher

native JavaFoil graph extraction engine: headless vector capture of Polar, Velocity/Cp, and Boundary Layer cards

'Extract JavaFoil Graphs (Native)' instant button in Configure tab and automated multi-artifact generation in Results tab